`views::concat`

Document #:	P2542R7
Date:	2023-09-23
Project:	Programming Language C++
Audience:	SG9, LEWG, LWG

Reply-to:	Hui Xie <hui.xie1990@gmail.com> S. Levent Yilmaz <levent.yilmaz@gmail.com>

Contents

- 1 Revision History
  - 1.1 R7
  - 1.2 R6
  - 1.3 R5
  - 1.4 R4
  - 1.5 R3
  - 1.6 R2
  - 1.7 R1
- 2 Abstract
- 3 Motivation and Examples
- 4 Design
  - 4.1 `concatable`-ity of ranges
    - 4.1.1 `reference`
    - 4.1.2 `value_type`
    - 4.1.3 `range_rvalue_reference_t`
    - 4.1.4 `indirectly_readable`
    - 4.1.5 Mixing ranges of references with ranges of prvalues
    - 4.1.6 Unsupported Cases and Potential Extensions for the Future
  - 4.2 Zero or one view
  - 4.3 Hidden *O(N)* time complexity for *N* adapted ranges
  - 4.4 Borrowed vs Cheap Iterator
  - 4.5 Common Range
    - 4.5.1 Discussion: Consolidate with `cartesian_product_view`
    - 4.5.2 `!common_range && (random_accessed_range && sized_range)`

— 4.6 Random Access Range
— 4.7 Sized Range
— 4.8 `iter_swap` Customizations
— 4.8.1 Option 1: Delegate to the Underlying `iter_swap`
— 4.8.2 Option 2: Do not Provide the Customization
— 4.8.3 Option 3: Delegate to the Underlying `iter_swap` Only If They Have the Same Type
— 4.9 Implementation Experience
— 5 Wording
— 5.1 Addition to `<ranges>`
— 5.2 Move exposition-only utilities
— 5.3 `concat`
— ?.?.? Concat view [range.concat]
— 5.4 Feature Test Macro
— 6 References

§ 1 Revision History

§ 1.1 R7

— update wording w.r.t `!common_range && random_access_range && sized_range` changes
— fix const-conversion constructor
— Various wording fixes

§ 1.2 R6

— Added a section `!common_range && random_access_range && sized_range`

§ 1.3 R5

— Removed `concat_expert` (reverting to R3) per SG9 poll.
— Fix `static_cast<difference_type>`
— Reuse utilities in `cartesian_product_view` to define `concat-is-bidirectional`
— Various wording fixes

§ 1.4 R4

— Add `concat_expert`

§ 1.5 R3

— Redesigned `iter_swap`
— Relaxed `random_access_range` constraints
— Fixed conversions of `difference` types
— Various wording fixes

1.6 R2

— Adding extra semantic constraints in the concept `concat-indirectly-readable` to prevent non-equality-preserving behaviour of `operator*` and `iter_move`.

1.7 R1

— Removed the `common_range` support for underlying ranges that are `!common_range && random_access_range && sized_range`.

— Introduced extra exposition concepts to simplify the wording that defines `concatable`.

2 Abstract

This paper proposes `views::concat` as very briefly introduced in Section 4.7 of [P2214R1]. It is a view factory that takes an arbitrary number of ranges as an argument list, and provides a view that starts at the first element of the first range, ends at the last element of the last range, with all range elements sequenced in between respectively in the order given in the arguments, effectively concatenating, or chaining together the argument ranges.

3 Motivation and Examples

Before

```
std::vector<int> v1{1,2,3}, v2{4,5}, v3{};
std::array  a{6,7,8};
int s = 9;
std::print("[{:n}, {:n}, {:n}, {:n}, {}]\n",
           v1, v2, v3, a, s);
// output:  [1, 2, 3, 4, 5, , 6, 7, 8, 9]
```

After

```
std::vector<int> v1{1,2,3}, v2{4,5}, v3{};
std::array  a{6,7,8};
auto s = std::views::single(9);
std::print("{}\n",
           std::views::concat(v1, v2, v3,
        a, s));
// output:  [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
class Foo;

class Bar{
public:
  const Foo& getFoo() const;
};

class MyClass{
  std::vector<Foo> foos_;
  Bar bar_;
public:
  auto getFoos () const{
    std::vector<std::reference_wrapper<Foo
        const>> fooRefs;
    fooRefs.reserve(foos_.size() + 1);

    for (auto const& f : foos_) {
      fooRefs.emplace_back(f);
    }
    fooRefs.emplace_back(bar_.getFoo());
    return fooRefs;
  }
};

// user
```

```
class Foo;

class Bar{
public:
  const Foo& getFoo() const;
};

class MyClass{
  std::vector<Foo> foos_;
  Bar bar_;
public:
  auto getFoos () const{
    return std::views::concat(
      foos_,
      std::views::single(std::cref(bar_))
      | std::views::transform(&Bar::getFoo)
    );
  }
};

// user
for(const auto& foo: myClass.getFoos()){
  // use foo, which is Foo const &
}
```

Before	After

`for(const auto& foo: myClass.getFoos()){`
`  // `foo` is std::reference_wrapper<Foo const>, not simply a Foo`
`  // ...`
`}`	

The first example shows that dealing with multiple ranges require care even in the simplest of cases: The "Before" version manually concatenates all the ranges in a formatting string, but empty ranges aren't handled and the result contains an extra comma and a space. With `concat_view`, empty ranges are handled per design, and the construct expresses the intent cleanly and directly.

In the second example, the user has a class composed of fragmented and indirect data. They want to implement a member function that provides a range-like view to all this data sequenced together in some order, and without creating any copies. The "Before" implementation is needlessly complex and creates a temporary container with a potentially problematic indirection in its value type. `concat_view` based implementation is a neat one-liner.

§ 4 Design

This is a generator factory as described in [P2214R1] Section 4.7. As such, it can not be piped to. It takes the list of ranges to concatenate as arguments to `ranges::concat_view` constructor, or to `ranges::views::concat` customization point object.

§ 4.1 `concatable`-ity of ranges

Adaptability of any given two or more distinct `ranges` into a sequence that itself models a `range`, depends on the compatibility of the reference and the value types of these ranges. A precise formulation is made in terms of `std::common_reference_t` and `std::common_type_t`, and is captured by the exposition only concept `concatable`. See Wording. Proposed `concat_view` is then additionally constrained by this concept. (Note that, this is an improvement over [range-v3] `concat_view` which lacks such constraints, and fails with hard errors instead.)

§ 4.1.1 `reference`

The `reference` type is the `common_reference_t` of all underlying range's `range_reference_t`. In addition, as the result of `common_reference_t` is not necessarily a reference type, an extra constraint is needed to make sure that each underlying range's `range_reference_t` is convertible to that common reference.

§ 4.1.2 `value_type`

To support the cases where underlying ranges have proxy iterators, such as `zip_view`, the `value_type` cannot simply be the `remove_cvref_t` of the `reference` type, and it needs to respect underlying ranges' `value_type`. Therefore, in this proposal the `value_type` is defined as the `common_type_t` of all underlying range's `range_value_t`.

§ 4.1.3 `range_rvalue_reference_t`

To make `concat_view`'s iterator's `iter_move` behave correctly for the cases where underlying iterators customise `iter_move`, such as `zip_view`, `concat_view` has to respect those customizations. Therefore, `concat_view` requires `common_reference_t` of all underlying ranges's `range_rvalue_reference_t` exist and can be converted to from each underlying range's `range_rvalue_reference_t`.

4.1.4 `indirectly_readable`

In order to make `concat_view` model `input_range`, `reference`, `value_type`, and `range_rvalue_reference_t` have to be constrained so that the iterator of the `concat_view` models `indirectly_readable`.

4.1.5 Mixing ranges of references with ranges of prvalues

In the following example,

```
std::vector<std::string> v = ...;
auto   r1   =   v   |   std::views::transform([](auto&&   s)   ->   std::string&&   {return
        std::move(s);});
auto r2 = std::views::iota(0, 2)
            | std::views::transform([](auto i){return std::to_string(i)});
auto cv = std::views::concat(r1, r2);
auto it = cv.begin();
*it; // first deref
*it; // second deref
```

`r1` is a range of which `range_reference_t` is `std::string&&`, while `r2`'s `range_reference_t` is `std::string`. The `common_reference_t` between these two `references` would be `std::string`. After the "first deref", even though the result is unused, there is a conversion from `std::string&&` to `std::string` when the result of underlying iterator's `operator*` is converted to the `common_reference_t`. This is a move construction and the underlying `vector`'s element is modified to a moved-from state. Later when the "second deref" is called, the result is a moved-from `std::string`. This breaks the "equality preserving" requirements.

Similar to `operator*`, `ranges::iter_move` has this same issue, and it is more common to run into this problem. For example,

```
std::vector<std::string> v = ...;
auto r = std::views::iota(0, 2)
          | std::views::transform([](auto i){return std::to_string(i)});
auto cv = std::views::concat(v, r);
auto it = cv.begin();
std::ranges::iter_move(it); // first iter_move
std::ranges::iter_move(it); // second iter_move
```

`v`'s `range_rvalue_reference_t` is `std::string&&`, and `r`'s `range_rvalue_reference_t` is `std::string`, the `common_reference_t` between them is `std::string`. After the first `iter_move`, the underlying `vector`'s first element is moved to construct the temporary `common_reference_t`, aka `std::string`. As a result, the second `iter_move` results in a moved-from state `std::string`. This breaks the "non-modifying" equality-preserving contract in `indirectly_readable` concept.

A naive solution for this problem is to ban all the usages of mixing ranges of references with ranges of prvalues. However, not all of this kind of mixing are problematic. For example, `concating` a range of `std::string&&` with a range of prvalue `std::string_view` is OK, because converting `std::string&&` to `std::string_view` does not modify the `std::string&&`. In general, it is not possible to detect whether the conversion from `T&&` to `U` modifies `T&&` through syntactic requirements. Therefore, the authors of this paper propose to use the "equality-preserving" semantic requirements of the requires-expression and the notational convention that constant lvalues shall not be modified in a manner observable to equality-preserving as defined in 18.2 [concepts.equality]. See Wording.

4.1.6 Unsupported Cases and Potential Extensions for the Future

Common type and reference based `concatable` logic is a practical and convenient solution that satisfies the motivation as outlined, and is what the authors propose in this paper. However, there are several potentially important use cases that get left out:

1. Concatenating ranges of different subclasses, into a range of their common base.
2. Concatenating ranges of unrelated types into a range of a user-determined common type, e.g. a `std::variant`.

Here is an example of the first case where `common_reference` formulation can manifest a rather counter-intuitive behavior: Let `D1` and `D2` be two types only related by their common base `B`, and `d1`, `d2`, and `b` be some range of these types, respectively. `concat(b, d1, d2)` is a well-formed range of `B` by the current formulation, suggesting such usage is supported. However, a mere reordering of the sequence, say to `concat(d1, d2, b)`, yields one that is not.

The authors believe that such cases should be supported, but can only be done so via an adaptor that needs at least one explicit type argument at its interface. A future extension may satisfy these use cases, for example a `concat_as` view, or by even generally via an `as` view that is a type-generalized version of the `as_const` view of [P2278R1].

§ 4.2 Zero or one view

— No argument `views::concat()` is ill-formed. It can not be a `views::empty<T>` because there is no reasonable way to determine an element type `T`.

— Single argument `views::concat(r)` is expression equivalent to `views::all(r)`, which intuitively follows.

§ 4.3 Hidden *O(N)* time complexity for *N* adapted ranges

Time complexities as required by the `ranges` concepts are formally expressed with respect to the total number of elements (the size) of a given range, and not to the statically known parameters of that range. Hence, the complexity of `concat_view` or its iterators' operations are documented to be constant time, even though some of these are a linear function of the number of ranges it concatenates which is a statically known parameter of this view.

Some examples of these operations for `concat_view` are copy, `begin` and `size`, and its iterators' increment, decrement, advance, distance. This characteristic (but not necessarily the specifics) are very much similar to the other n-ary adaptors like `zip_view` [P2321R2] and `cartesian_view` [P2374R3].

§ 4.4 Borrowed vs Cheap Iterator

`concat_view` can be designed to be a `borrowed_range`, if all the underlying ranges are. However, this requires the iterator implementation to contain a copy of all iterators and sentinels of all underlying ranges at all times (just like that of `views::zip` [P2321R2]). On the other hand (unlike `views::zip`), a much cheaper implementation can satisfy all the proposed functionality provided it is permitted to be unconditionally not borrowed. This implementation would maintain only a single active iterator at a time and simply refers to the parent view for the bounds.

Experience shows the borrowed-ness of `concat` is not a major requirement; and the existing implementation in [range-v3] seems to have picked the cheaper alternative. This paper proposes the same.

§ 4.5 Common Range

`concat_view` can be `common_range` if the last underlying range models `common_range`

4.5.1 Discussion: Consolidate with `cartesian_product_view`

In the R0 version, `concat_view` can be `common_range` if the last underlying range models `common_range || (random_access_range && sized_range)`

The `end` function was defined as

```
if constexpr (common_range<last-view>) {
    constexpr auto N = sizeof...(Views);
    return iterator<is-const>{this, in_place_index<N - 1>,
                      ranges::end(get<N - 1>(views_))};
} else if constexpr (random_access_range<last-view> && sized_range<last-view>) {
    constexpr auto N = sizeof...(Views);
    return iterator<is-const>{
        this, in_place_index<N - 1>,
        ranges::begin(get<N - 1>(views_)) + ranges::size(get<N - 1>(views_))};
} else {
    return default_sentinel;
}
```

Following SG9's direction, the `random_access_range && sized_range` was removed in R1 version for simplicity

```
if constexpr (common_range<last-view>) {
    constexpr auto N = sizeof...(Views);
    return iterator<is-const>{this, in_place_index<N - 1>,
                      ranges::end(get<N - 1>(views_))};
} else {
    return default_sentinel;
}
```

However, `cartesian_product_view` 26.7.32.2 [range.cartesian.view] defines its `end` function in a way that is very similar to `concat_view`'s R0 version.

```
  template<class R>
  concept cartesian-product-common-arg =                // exposition only
    common_range<R> || (sized_range<R> && random_access_range<R>);

  template<class First, class... Vs>
  concept cartesian-product-is-common =                 // exposition only
    cartesian-product-common-arg<First>;

  template<cartesian-product-common-arg R>
  constexpr auto cartesian-common-arg-end(R& r) {       // exposition only
    if constexpr (common_range<R>) {
      return ranges::end(r);
    } else {
      return ranges::begin(r) + ranges::distance(r);
    }
  }

  constexpr iterator<false> end()
      requires ((!simple-view<First> || ... || !simple-view<Vs>) &&
        cartesian-product-is-common<First, Vs...>);
  constexpr iterator<true> end() const
      requires cartesian-product-is-common<const First, const Vs...>;

  constexpr default_sentinel_t end() const noexcept;
```

For consistency between `cartesian_product_view` and `concat_view`, it would be good to add back the support of `random_access_range && size_range`. Those exposition only concepts and functions can be reused for both of the views, and their definitions of `end` function would be very similar, except that

`cartesian_product_view` checks the first underlying range and `concat_view` checks the last underlying range.

As per SG9 direction, this section is dropped.

4.5.2 `!common_range && (random_accessed_range && sized_range)`

In R0 version, `concat_view` is a `common_range` if the last range satisfies `common_range || (random_accessed_range && sized_range)`

As per SG9's direction, `concat_view` should not pretend to be a `common_range`, when the last underlying range is a `!common_range && random_accessed_range && sized_range`, even though the end iterator could be computed from `ranges::begin(r) + ranges::size(r)`.

Therefore in the current version the `common_range` is simplified to,

```
concat-is-common = common_range<last-view>
```

However, the current design of `concat_view`'s bidirectional behaviour does use `ranges::begin(r) + ranges::size(r) - 1` to get to the previous range's last iterator, when the previous range is `!common_range && random_accessed_range && sized_range`. Its `bidirectional_range` requirement is designed as follows

```
template <class R>
concept common-ish = common_range<R> || (random_accessed_range<R> && sized_range<R>)

concat-is-bidi = (bidirectional_range<all-views> && ...) && (common-ish<all-but-last-
        views> && ...)
```

In the LWG wording review on 2023-09-13, it was suggested that `concat_view`'s `bidirectional_range` behaviour should be consistent with its `common_range` behaviour, i.e, do not support `!common_range && random_accessed_range && sized_range` ranges. This would simplify the wording, so that to get to the end of the previous range, `--ranges::end(r)` would always work.

So,

```
concat-is-bidi = (bidirectional_range<all-views> && ...) && (common_range<all-but-last-
        views> && ...)
```

Similarly, `random_access_range` could follow the same design. To implement `it + n`, it is necessary to know if `n` is larger than the distance from `it` to the end of the current underlying range. If the underlying range is `!common_range && random_accessed_range && sized_range`, to compute the distance from `it` to the end of the range, an implementation must do `ranges::size(r) - (it - ranges::begin(r))`. If it is `common_range`, the distance is simply `ranges::distance(it, ranges::end(r))`.

If the support of `!common_range && random_accessed_range && sized_range` is dropped, it would simplify the exposition only helper functions `advance_fwd`, `advance_bwd` and all other random access related functions.

So change

```
concat-is-random-access = (random_access_range<all-views> && ...) && (sized_range<all-
        but-last-views> && ...)
```

to

```
concat-is-random-access = (random_access_range<all-views> && ...) && (common_range<all-
        but-last-views> && ...)
```

Ranges that are `!common_range && random_accessed_range && sized_range` are rare, and if the user does have one of these ranges and would like to have common, bidirectional or random access support in the `concat_view`, they can do

```
auto cv = views::concat(r | views::common, other_views...)
```

As per LEWG direction, the change proposed in this section is adopted ## Bidirectional Range

`concat_view` can model `bidirectional_range` if the underlying ranges satisfy the following conditions:

- Every underlying range models `bidirectional_range`
- If the iterator is at nth range's begin position, after `operator--` it should go to (n-1)th's range's end-1 position. This means, the (n-1)th range has to support either
  - `common_range && bidirectional_range`, so the position can be reached by `--ranges::end(n-1th range)`, assuming n-1th range is not empty, or
  - `random_access_range && sized_range`, so the position can be reached by `ranges::begin(n-1th range) + (ranges::size(n-1th range) - 1)` in constant time, assuming n-1th range is not empty. However, as per LEWG's direction,`!common_range && random_access_range && sized_range` is removed

  Note that the last underlying range does not have to satisfy this constraint because n-1 can never be the last underlying range. If neither of the two constraints is satisfied, in theory we can cache the end-1 position for every single underlying range inside the `concat_view` itself. But the authors do not consider this type of ranges as worth supporting bidirectional

In the `concat` implementation in [range-v3], `operator--` is only constrained on all underlying ranges being `bidirectional_range` on the declaration, but its implementation is using `ranges::next(ranges::begin(r), ranges::end(r))` which implicitly requires random access to make the operation constant time. So it went with the second constraint.

§ 4.6 Random Access Range

`concat_view` can model `random_access_range` if the underlying ranges satisfy the following conditions:

- Every underlying range models `random_access_range`
- All except the last range model `common_range`

§ 4.7 Sized Range

`concat_view` can be `sized_range` if all the underlying ranges model `sized_range`

§ 4.8 `iter_swap` Customizations

§ 4.8.1 Option 1: Delegate to the Underlying `iter_swap`

This option was originally adopted in the paper. If all the combinations of the underlying iterators model `indirectly_swappable`, then `ranges::iter_swap(x, y)` could delegate to the underlying iterators

```
std::visit(ranges::iter_swap, x.it_, y.it_);
```

This would allow swapping elements across heterogeneous ranges, which is very powerful. However, `ranges::iter_swap` is too permissive. Consider the following example:

```
int v1[] = {1, 2};
long v2[] = {2147483649, 4};
```

```
auto cv = std::views::concat(v1, v2);
auto it1 = cv.begin();
auto it2 = cv.begin() + 2;
std::ranges::iter_swap(it1, it2);
```

Delegating `ranges::iter_swap(const concat_view::iterator&, const concat_view::iterator&)` to `ranges::iter_swap(int*, long*)` in this case would result in a tripple-move and leave `v1[0]` overflowed.

Another example discussed in SG9 mailing list is

```
std::string_view v1[] = {"aa"};
std::string v2[] = {"bbb"};
auto cv = std::views::concat(v1, v2);
auto it1 = cv.begin();
auto it2 = cv.begin()+1;
std::ranges::iter_swap(it1, it2);
```

`ranges::iter_swap(string_view*, string*)` in this case would result in dangling reference due to the tripple move, which is effectively

```
void iter_swap_impl(string_view* x, string* y)
{
  std::string tmp(std::move(*y));
  *y = std::move(*x);
  *x = std::move(tmp); // *x points to local string tmp
}
```

It turned out that allowing swapping elements across heterogeneous ranges could result in lots of undesired behaviours.

4.8.2 Option 2: Do not Provide the Customization

If the `iter_swap` customization is removed, the above examples are no longer an issue because `ranges::iter_swap` would be ill-formed. This is because `iter_reference_t<concat_view::iterator>` are prvalues in those cases.

For the trivial cases where underlying ranges share the same `iter_reference_t`, `iter_value_t` and `iter_rvalue_reference_t`, the genereated tripple-move swap would just work.

However, all the user `iter_swap` customizations will be ignored, even if the user tries to `concat` the same type of ranges with `iter_swap` customizations.

4.8.3 Option 3: Delegate to the Underlying `iter_swap` Only If They Have the Same Type

This option was suggested by Tomasz in the SG9 mailing list. The idea is to

— 1. use `ranges::iter_swap(x.it_, y.it_)` if they have the same underlying iterator type

— 2. otherwise, `ranges::swap(*x, *y)` if it is valid

The issues decribed in Option 1 are avoided because we only delegate to underlying `iter_swap` if they have the same underlying iterators.

The authors believe that this apporach is a good compromise thus it is adopted in this revision.

4.9 Implementation Experience

`views::concat` has been implemented in [range-v3], with equivalent semantics as proposed here. The authors also implemented a version that directly follows the proposed wording below without any issue [ours].

5 Wording

5.1 Addition to `<ranges>`

Add the following to 26.2 [ranges.syn], header `<ranges>` synopsis:

```
// [...]
namespace std::ranges {
  // [...]

  // [range.concat], concat view
  template <input_range... Views>
    requires see below
  class concat_view;

  namespace views {
    inline constexpr unspecified concat = unspecified;
  }

}
```

5.2 Move exposition-only utilities

Move *`all-random-access`*, *`all-bidirectional`*, and *`all-forward`* from 26.7.24.3 [range.zip.iterator] to 26.7.5 [range.adaptor.helpers].

Add the following to 26.7.5 [range.adaptor.helpers]

```
namespace std::ranges {
// [...]
+ template<bool Const, class... Views>
+   concept all-random-access =                 // exposition only
+     (random_access_range<maybe-const<Const, Views>> && ...);
+ template<bool Const, class... Views>
+   concept all-bidirectional =                 // exposition only
+     (bidirectional_range<maybe-const<Const, Views>> && ...);
+ template<bool Const, class... Views>
+   concept all-forward =                       // exposition only
+     (forward_range<maybe-const<Const, Views>> && ...);
// [...]
}
```

Remove the following from 26.7.24.3 [range.zip.iterator]

```
namespace std::ranges {
- template<bool Const, class... Views>
-   concept all-random-access =                 // exposition only
-     (random_access_range<maybe-const<Const, Views>> && ...);
- template<bool Const, class... Views>
-   concept all-bidirectional =                 // exposition only
-     (bidirectional_range<maybe-const<Const, Views>> && ...);
- template<bool Const, class... Views>
-   concept all-forward =                       // exposition only
-     (forward_range<maybe-const<Const, Views>> && ...);
// [...]
}
```

5.3 `concat`

Add the following subclause to 26.7 [range.adaptors].

?.?.? Concat view [range.concat]

?.?.?.1 Overview [range.concat.overview]

1 `concat_view` presents a `view` that concatenates all the underlying ranges.

2 The name `views::concat` denotes a customization point object (16.3.3.3.5 [customization.point.object]). Given a pack of subexpressions `Es...`, the expression `views::concat(Es...)` is expression-equivalent to

(2.1) — `views::all(Es...)` if `Es` is a pack with only one element and `views::all(Es...)` is a well formed expression,

(2.2) — otherwise, `concat_view(Es...)`

[*Example:*

```
std::vector<int> v1{1,2,3}, v2{4,5}, v3{};
std::array  a{6,7,8};
auto s = std::views::single(9);
for(auto&& i : std::views::concat(v1, v2, v3, a, s)){
  std::cout << i << ' '; // prints: 1 2 3 4 5 6 7 8 9
}
```

— *end example*]

?.?.?.2 Class template `concat_view` [range.concat.view]

```
namespace std::ranges {

  template <class... Rs>
  using concat-reference-t = common_reference_t<range_reference_t<Rs>...>; // exposition
        only

  template <class... Rs>
  using concat-value-t = common_type_t<range_value_t<Rs>...>; // exposition only

  template <class... Rs>
  using concat-rvalue-reference-t = common_reference_t<range_rvalue_reference_t<Rs>...>;
        // exposition only

  template <class... Rs>
  concept concat-indirectly-readable = see below; // exposition only

  template <class... Rs>
  concept concatable = see below;             // exposition only

  template <bool Const, class... Rs>
  concept concat-is-random-access = see below;   // exposition only

  template <bool Const, class... Rs>
  concept concat-is-bidirectional = see below;   // exposition only

  template <input_range... Views>
    requires (view<Views> && ...) && (sizeof...(Views) > 0) &&
             concatable<Views...>
  class concat_view : public view_interface<concat_view<Views...>> {
    tuple<Views...> views_;                   // exposition only
```

```
    template <bool Const>
    class iterator;                          // exposition only

  public:
    constexpr concat_view() = default;

    constexpr explicit concat_view(Views... views);

    constexpr iterator<false> begin() requires(!(simple-view<Views> && ...));

    constexpr iterator<true> begin() const
      requires((range<const Views> && ...) && concatable<const Views...>);

    constexpr auto end() requires(!(simple-view<Views> && ...));

    constexpr auto end() const
      requires((range<const Views> && ...) && concatable<const Views...>);

    constexpr auto size() requires(sized_range<Views>&&...);

    constexpr auto size() const requires(sized_range<const Views>&&...);
  };

  template <class... R>
    concat_view(R&&...) -> concat_view<views::all_t<R>...>;
}
```

```
template <class... Rs>
concept concat-indirectly-readable = see below; // exposition only
```

1 The exposition-only *concat-indirectly-readable* concept is equivalent to:

```
template <class Ref, class RRef, class It>
concept concat-indirectly-readable-impl =  // exposition only
requires (const It it) {
  { *it } -> convertible_to<Ref>;
  { ranges::iter_move(it) } -> convertible_to<RRef>;
};

template <class... Rs>
concept concat-indirectly-readable = // exposition only
  common_reference_with<concat-reference-t<Rs...>&&,
                        concat-value-t<Rs...>&> &&
  common_reference_with<concat-reference-t<Rs...>&&,
                        concat-rvalue-reference-t<Rs...>&&> &&
  common_reference_with<concat-rvalue-reference-t<Rs...>&&,
                        concat-value-t<Rs...> const&> &&
  (concat-indirectly-readable-impl<concat-reference-t<Rs...>,
                                   concat-rvalue-reference-t<Rs...>,
                                   iterator_t<Rs>> && ...);
```

```
template <class... Rs>
concept concatable = see below; // exposition only
```

2 The exposition-only *concatable* concept is equivalent to:

```
template <class... Rs>
concept concatable = requires { // exposition only
  typename concat-reference-t<Rs...>;
  typename concat-value-t<Rs...>;
  typename concat-rvalue-reference-t<Rs...>;
} && concat-indirectly-readable<Rs...>;
```

```
template <bool Const, class... Rs>
concept concat-is-random-access = see below; // exposition only
```

3 Let `Fs` be the pack that consists of all elements of `Rs` except the last element, then *`concat-is-random-access`* is equivalent to:

```
template <bool Const, class... Rs>
concept concat-is-random-access = // exposition only
   all-random-access<Const, Rs...> &&
   (common_range<maybe-const<Const, Fs>> && ...);
```

```
template <bool Const, class... Rs>
concept concat-is-bidirectional = see below; // exposition only
```

4 Let `Fs` be the pack that consists of all elements of `Rs` except the last element, then *`concat-is-bidirectional`* is equivalent to:

```
template <bool Const, class... Rs>
concept concat-is-bidirectional = // exposition only
   (all-bidirectional<Const, Rs...> && ... && common_range<maybe-const<Const,
        Fs>>);
```

```
constexpr explicit concat_view(Views... views);
```

5 *Effects*: Initializes *`views_`* with `std::move(views)...`.

```
constexpr iterator<false> begin() requires(!(simple-view<Views> && ...));
constexpr iterator<true> begin() const
  requires((range<const Views> && ...) && concatable<const Views...>);
```

6 *Effects*: Let *`is-const`* be `true` for the const-qualified overload, and `false` otherwise. Equivalent to:

```
iterator<is-const> it(this, in_place_index<0>, ranges::begin(get<0>(views_)));
it.template satisfy<0>();
return it;
```

```
constexpr auto end() requires(!(simple-view<Views> && ...));
constexpr auto end() const
  requires((range<const Views> && ...) && concatable<const Views...>);
```

7 *Effects*: Let *`is-const`* be `true` for the const-qualified overload, and `false` otherwise, and let *`last-view`* be the last element of the pack `const Views...` for the const-qualified overload, and the last element of the pack `Views...` otherwise. Equivalent to:

```
if constexpr (common_range<last-view>) {
    constexpr auto N = sizeof...(Views);
    return iterator<is-const>(this, in_place_index<N - 1>,
                      ranges::end(get<N - 1>(views_)));
} else {
    return default_sentinel;
}
```

```
constexpr auto size() requires(sized_range<Views>&&...);
constexpr auto size() const requires(sized_range<const Views>&&...);
```

8 *Effects*: Equivalent to:

```
return apply(
    [](auto... sizes) {
        using CT = make-unsigned-like-t<common_type_t<decltype(sizes)...>>;
        return (CT(sizes) + ...);
```

```
    },
    tuple-transform(ranges::size, views_));
```

?.?.?.3 Class concat_view::iterator [range.concat.iterator]

```
namespace std::ranges{

  template <input_range... Views>
    requires (view<Views> && ...) && (sizeof...(Views) > 0) &&
             concatable<Views...>
  template <bool Const>
  class concat_view<Views...>::iterator {

  public:
    using iterator_category = see below;                    // not always present.
    using iterator_concept = see below;
    using value_type = concat-value-t<maybe-const<Const, Views>...>;
         using  difference_type  =  common_type_t<range_difference_t<maybe-const<Const,
        Views>>...>;

  private:
    using base-iter =                                       // exposition only
      variant<iterator_t<maybe-const<Const, Views>>...>;

    maybe-const<Const, concat_view>* parent_ = nullptr;   // exposition only
    base-iter it_;                                          // exposition only

    template <std::size_t N>
    constexpr void satisfy();                               // exposition only

    template <std::size_t N>
    constexpr void prev();                                  // exposition only

    template <std::size_t N>
       constexpr  void  advance-fwd(difference_type  offset,  difference_type  steps);  //
        exposition only

    template <std::size_t N>
       constexpr  void  advance-bwd(difference_type  offset,  difference_type  steps);  //
        exposition only

    template <class... Args>
    explicit constexpr iterator(maybe-const<Const, concat_view>* parent, Args&&... args)
        requires constructible_from<base-iter, Args&&...>;  // exposition only

  public:

    iterator() = default;

    constexpr iterator(iterator<!Const> i)
        requires Const && (convertible_to<iterator_t<Views>, iterator_t<const Views>> &&
        ...);

    constexpr decltype(auto) operator*() const;

    constexpr iterator& operator++();

    constexpr void operator++(int);

    constexpr iterator operator++(int)
        requires all-forward<Const, Views...>;

    constexpr iterator& operator--()
        requires concat-is-bidirectional<Const, Views...>;
```

```
    constexpr iterator operator--(int)
        requires concat-is-bidirectional<Const, Views...>;

    constexpr iterator& operator+=(difference_type n)
        requires concat-is-random-access<Const, Views...>;

    constexpr iterator& operator-=(difference_type n)
        requires concat-is-random-access<Const, Views...>;

    constexpr decltype(auto) operator[](difference_type n) const
        requires concat-is-random-access<Const, Views...>;

    friend constexpr bool operator==(const iterator& x, const iterator& y)
        requires(equality_comparable<iterator_t<maybe-const<Const, Views>>>&&...);

    friend constexpr bool operator==(const iterator& it, default_sentinel_t);

    friend constexpr bool operator<(const iterator& x, const iterator& y)
        requires all-random-access<Const, Views...>;

    friend constexpr bool operator>(const iterator& x, const iterator& y)
        requires all-random-access<Const, Views...>;

    friend constexpr bool operator<=(const iterator& x, const iterator& y)
        requires all-random-access<Const, Views...>;

    friend constexpr bool operator>=(const iterator& x, const iterator& y)
        requires all-random-access<Const, Views...>;

    friend constexpr auto operator<=>(const iterator& x, const iterator& y)
        requires (all-random-access<Const, Views...> &&
         (three_way_comparable<iterator_t<maybe-const<Const, Views>>> && ...));

    friend constexpr iterator operator+(const iterator& it, difference_type n)
        requires concat-is-random-access<Const, Views...>;

    friend constexpr iterator operator+(difference_type n, const iterator& it)
        requires concat-is-random-access<Const, Views...>;

    friend constexpr iterator operator-(const iterator& it, difference_type n)
        requires concat-is-random-access<Const, Views...>;

    friend constexpr difference_type operator-(const iterator& x, const iterator& y)
        requires concat-is-random-access<Const, Views...>;

    friend constexpr difference_type operator-(const iterator& x, default_sentinel_t)
        requires see below;

    friend constexpr difference_type operator-(default_sentinel_t, const iterator& x)
        requires see below;

    friend constexpr decltype(auto) iter_move(const iterator& it) noexcept(see below);

     friend constexpr void iter_swap(const iterator& x, const iterator& y) noexcept(see
        below)
        requires see below;
  };

}
```

1 *iterator*::iterator_concept is defined as follows:

(1.1) — If *concat-is-random-access*<Const, Views...> is modeled, then iterator_concept denotes random_access_iterator_tag.

(1.2) — Otherwise, if *concat-is-bidirectional*<Const, Views...> is modeled, then iterator_concept denotes bidirectional_iterator_tag.

(1.3) — Otherwise, if *all-forward*<Const, Views...> is modeled, then iterator_concept denotes forward_iterator_tag.

(1.4) — Otherwise, iterator_concept denotes input_iterator_tag.

2 The member typedef-name iterator_category is defined if and only if *all-forward*<Const, Views...> is modeled. In that case, iterator::iterator_category is defined as follows:

(2.1) — If is_reference_v<*concat-reference-t*<*maybe-const*<Const, Views>...>> is false, then iterator_category denotes input_iterator_tag

(2.2) — Otherwise, let Cs denote the pack of types iterator_traits<iterator_t<*maybe-const*<Const, Views>>>::iterator_category....

(2.2.1) — If (derived_from<Cs, random_access_iterator_tag> && ...) && *concat-is-random-access*<Const, Views...> is true, iterator_category denotes random_access_iterator_tag.

(2.2.2) — Otherwise, if (derived_from<Cs, bidirectional_iterator_tag> && ...) && *concat-is-bidirectional*<Const, Views...> is true, iterator_category denotes bidirectional_iterator_tag.

(2.2.3) — Otherwise, If (derived_from<Cs, forward_iterator_tag> && ...) is true, iterator_category denotes forward_iterator_tag.

(2.2.4) — Otherwise, iterator_category denotes input_iterator_tag.

```
template <std::size_t N>
constexpr void satisfy();                              // exposition only
```

3 *Effects*: Equivalent to:

```
if constexpr (N < (sizeof...(Views) - 1)) {
    if (get<N>(it_) == ranges::end(get<N>(parent_->views_))) {
            it_.template emplace<N + 1>(ranges::begin(get<N + 1>(parent_-
        >views_)));
        satisfy<N + 1>();
    }
}
```

```
template <std::size_t N>
constexpr void prev();                                 // exposition only
```

4 *Effects*: Equivalent to:

```
if constexpr (N == 0) {
    --get<0>(it_);
} else {
    if (get<N>(it_) == ranges::begin(get<N>(parent_->views_))) {
        it_.template emplace<N - 1>(ranges::end(get<N - 1>(parent_->views_)));
        prev<N - 1>();
    } else {
        --get<N>(it_);
    }
}
```

```
template <std::size_t N>
constexpr void advance-fwd(difference_type offset, difference_type steps); // exposition
        only
```

5 *Effects*: Equivalent to:

```
using underlying_diff_type = iter_difference_t<variant_alternative_t<N, base-
        iter>>;
if constexpr (N == sizeof...(Views) - 1) {
    get<N>(it_) += static_cast<underlying_diff_type>(steps);
} else {
    auto n_size = ranges::distance(get<N>(parent_->views_));
    if (offset + steps < n_size) {
        get<N>(it_) += static_cast<underlying_diff_type>(steps);
    } else {
            it_.template emplace<N + 1>(ranges::begin(get<N + 1>(parent_-
        >views_)));
        advance-fwd<N + 1>(0, offset + steps - n_size);
    }
}
```

```
template <std::size_t N>
constexpr void advance-bwd(difference_type offset, difference_type steps); // exposition
        only
```

6 *Effects*: Equivalent to:

```
using underlying_diff_type = iter_difference_t<variant_alternative_t<N, base-
        iter>>;
if constexpr (N == 0) {
    get<N>(it_) -= static_cast<underlying_diff_type>(steps);
} else {
    if (offset >= steps) {
        get<N>(it_) -= static_cast<underlying_diff_type>(steps);
    } else {
        auto prev_size = ranges::distance(get<N - 1>(parent_->views_));
        it_.template emplace<N - 1>(ranges::end(get<N - 1>(parent_->views_)));
        advance-bwd<N - 1>(prev_size, steps - offset);
    }
}
```

```
template <class... Args>
explicit constexpr iterator(
           maybe-const<Const, concat_view>* parent,
           Args&&... args)
   requires constructible_from<base-iter, Args&&...>; // exposition only
```

7 *Effects*: Initializes *parent_* with `parent`, and initializes *it_* with `std::forward<Args>(args)...`.

```
constexpr iterator(iterator<!Const> it)
   requires Const &&
   (convertible_to<iterator_t<Views>, iterator_t<const Views>> && ...);
```

8 *Effects*: Initializes *parent_* with `it.`*parent_*, and let *i* be `it.`*it_*`.index()`, initializes *it_* with *base-iter*`(in_place_index<`*i*`>, get<`*i*`>(std::move(it.`*it_*`)))`

```
constexpr decltype(auto) operator*() const;
```

9 *Preconditions*: *it_*`.valueless_by_exception()` is `false`.

10 *Effects*: Equivalent to:

```
    using reference = concat-reference-t<maybe-const<Const, Views>...>;
    return std::visit([](auto&& it) -> reference {
        return *it; }, it_);
```

```
constexpr iterator& operator++();
```

11 *Preconditions*: `it_.valueless_by_exception()` is `false`.

12 *Effects*: Let `i` be `it_.index()`. Equivalent to:

```
    ++get<i>(it_);
    satisfy<i>();
    return *this;
```

```
constexpr void operator++(int);
```

13 *Effects*: Equivalent to:

```
    ++*this;
```

```
constexpr iterator operator++(int)
    requires all-forward<Const, Views...>;
```

14 *Effects*: Equivalent to:

```
    auto tmp = *this;
    ++*this;
    return tmp;
```

```
constexpr iterator& operator--()
    requires concat-is-bidirectional<Const, Views...>;
```

15 *Preconditions*: `it_.valueless_by_exception()` is `false`.

16 *Effects*: Let `i` be `it_.index()`. Equivalent to:

```
    prev<i>();
    return *this;
```

```
constexpr iterator operator--(int)
    requires concat-is-bidirectional<Const, Views...>;
```

17 *Effects*: Equivalent to:

```
    auto tmp = *this;
    --*this;
    return tmp;
```

```
constexpr iterator& operator+=(difference_type n)
    requires concat-is-random-access<Const, Views...>;
```

18 *Preconditions*: `it_.valueless_by_exception()` is `false`.

19 *Effects*: Let `i` be `it_.index()`. Equivalent to:

```
    if(n > 0) {
      advance-fwd<i>(get<i>(it_) - ranges::begin(get<i>(parent_->views_)), n);
    } else if (n < 0) {
      advance-bwd<i>(get<i>(it_) - ranges::begin(get<i>(parent_->views_)), -n);
    }
    return *this;
```

```
constexpr iterator& operator-=(difference_type n)
    requires concat-is-random-access<Const, Views...>;
```

20 *Effects*: Equivalent to:

```
*this += -n;
return *this;
```

```
constexpr decltype(auto) operator[](difference_type n) const
    requires concat-is-random-access<Const, Views...>;
```

21 *Effects*: Equivalent to:

```
return *((*this) + n);
```

```
friend constexpr bool operator==(const iterator& x, const iterator& y)
    requires(equality_comparable<iterator_t<maybe-const<Const, Views>>>&&...);
```

22 *Preconditions*: `x.it_.valueless_by_exception()` and `y.it_.valueless_by_exception()` are each `false`.

23 *Effects*: Equivalent to:

```
return x.it_ == y.it_;
```

```
friend constexpr bool operator==(const iterator& it, default_sentinel_t);
```

24 *Preconditions*: `it.it_.valueless_by_exception()` is `false`.

25 *Effects*: Equivalent to:

```
constexpr auto last_idx = sizeof...(Views) - 1;
return it.it_.index() == last_idx &&
              get<last_idx>(it.it_)  ==  ranges::end(get<last_idx>(it.parent_-
       >views_));
```

```
friend constexpr bool operator<(const iterator& x, const iterator& y)
    requires all-random-access<Const, Views...>;
friend constexpr bool operator>(const iterator& x, const iterator& y)
    requires all-random-access<Const, Views...>;
friend constexpr bool operator<=(const iterator& x, const iterator& y)
    requires all-random-access<Const, Views...>;
friend constexpr bool operator>=(const iterator& x, const iterator& y)
    requires all-random-access<Const, Views...>;
friend constexpr auto operator<=>(const iterator& x, const iterator& y)
   requires (all-random-access<Const, Views...> &&
    (three_way_comparable<iterator_t<maybe-const<Const, Views>>> && ...));
```

26 *Preconditions*: `x.it_.valueless_by_exception()` and `y.it_.valueless_by_exception()` are each `false`.

27 Let *op* be the operator.

28 *Effects*: Equivalent to:

```
return x.it_ op y.it_;
```

```
friend constexpr iterator operator+(const iterator& it, difference_type n)
    requires concat-is-random-access<Const, Views...>;
```

29 *Preconditions*: `it.it_.valueless_by_exception()` is `false`.

30 *Effects*: Equivalent to:

```
return iterator(it) += n;
```

```
friend constexpr iterator operator+(difference_type n, const iterator& it)
    requires concat-is-random-access<Const, Views...>;
```

31 *Effects*: Equivalent to:

```
return it + n;
```

```
friend constexpr iterator operator-(const iterator& it, difference_type n)
    requires concat-is-random-access<Const, Views...>;
```

32 *Effects*: Equivalent to:

```
return iterator(it) -= n;
```

```
friend constexpr difference_type operator-(const iterator& x, const iterator& y)
    requires concat-is-random-access<Const, Views...>;
```

33 *Preconditions*: `x.it_.valueless_by_exception()` and `y.it_.valueless_by_exception()` are each `false`.

34 *Effects*: Let i_x denote `x.it_.index()` and i_y denote `y.it_.index()`

(34.1) — if $i_x > i_y$, let d_y be `ranges::distance(get<`i_y`>(y.it_), ranges::end(get<`i_y`>(y.`*parent_*`->`*views_*`)))`, d_x be `ranges::distance(ranges::begin(get<`i_x`>(x.`*parent_*`->`*views_*`)), get<`i_x`>(x.`*it_*`))`. Let `s` denote the sum of the sizes of all the ranges `get<`*i*`>(x.`*parent_*`->`*views_*`)` for every integer $i_y < i < i_x$ if there is any, and `0` otherwise, of type `difference_type`, equivalent to

```
return d_y + s + d_x;
```

(34.2) — otherwise, if $i_x < i_y$, equivalent to:

```
return -(y - x);
```

(34.3) — otherwise, equivalent to:

```
return get<i_x>(x.it_) - get<i_y>(y.it_);
```

```
friend constexpr difference_type operator-(const iterator& x, default_sentinel_t)
    requires see below;
```

35 *Preconditions*: `x.it_.valueless_by_exception()` is `false`.

36 *Effects*: Let i_x denote `x.it_.index()`, d_x be `ranges::distance(get<`i_x`>(x.it_), ranges::end(get<`i_x`>(x.`*parent_*`->`*views_*`)))`. Let `s` denote the sum of the sizes of all the ranges `get<`*i*`>(x.`*parent_*`->`*views_*`)` for every integer $i_x < i <$ `sizeof...(Views)` if there is any, and `0` otherwise, of type `difference_type`, equivalent to

```
return -(d_x + s);
```

37 *Remarks*: Let `V` be the last element of the pack `Views`, the expression in the requires-clause is equivalent to:

```
(concat-is-random-access<Const, Views...> && common_range<maybe-const<Const,
        V>>)
```

```
friend constexpr difference_type operator-(default_sentinel_t, const iterator& x)
    requires see below;
```

38 *Effects*: Equivalent to:

```
return -(x - default_sentinel);
```

39 *Remarks*: Let `V` be the last element of the pack `Views`, the expression in the requires-clause is equivalent to:

```
(concat-is-random-access<Const, Views...> && common_range<maybe-const<Const,
        V>>)
```

```
friend constexpr decltype(auto) iter_move(const iterator& it) noexcept(see below);
```

40 *Preconditions*: `it.it_.valueless_by_exception()` is `false`.

41 *Effects*: Equivalent to:

```
return std::visit(
    [](const auto& i) ->
        concat-rvalue-reference-t<maybe-const<Const, Views>...> {
        return ranges::iter_move(i);
    },
    it.it_);
```

42 *Remarks*: The exception specification is equivalent to:

```
((is_nothrow_invocable_v<decltype(ranges::iter_move),
                         const iterator_t<maybe-const<Const, Views>>&> &&
      is_nothrow_convertible_v<range_rvalue_reference_t<maybe-const<Const,
        Views>>,
                               concat-rvalue-reference-t<maybe-const<Const,
        Views>...>>) && ...)
```

```
friend constexpr void iter_swap(const iterator& x, const iterator& y) noexcept(see
        below)
    requires see below;
```

43 *Preconditions*: `x.it_.valueless_by_exception()` and `y.it_.valueless_by_exception()` are each `false`.

44 *Effects*: Equivalent to:

```
std::visit(
    [&](const auto& it1, const auto& it2) {
        if constexpr (is_same_v<decltype(it1), decltype(it2)>) {
            ranges::iter_swap(it1, it2);
        } else {
            ranges::swap(*x, *y);
        }
    },
    x.it_, y.it_);
```

45 *Remarks*: The exception specification is equivalent to

```
(noexcept(ranges::swap(*x, *y)) && ... && noexcept(ranges::iter_swap(its,
        its)))
```

where `its` is a pack of lvalues of type `const iterator_t<maybe-const<Const, Views>>` respectively.

46 *Remarks*: The expression in the requires-clause is equivalent to

```
swappable_with<iter_reference_t<iterator>, iter_reference_t<iterator>> &&
(... && indirectly_swappable<iterator_t<maybe-const<Const, Views>>>)
```

§ 5.4 Feature Test Macro

Add the following macro definition to 17.3.2 [version.syn], header `<version>` synopsis, with the value selected by the editor to reflect the date of adoption of this paper:

```
#define __cpp_lib_ranges_concat  20XXXXL // also in <ranges>
```

6 References

[ours] Hui Xie and S. Levent Yilmaz. A proof-of-concept implementation of views::concat.
https://github.com/huixie90/cpp_papers/tree/main/impl/concat

[P2214R1] Barry Revzin, Conor Hoekstra, Tim Song. 2021-09-14. A Plan for C++23 Ranges.
https://wg21.link/p2214r1

[P2278R1] Barry Revzin. 2021-09-15. cbegin should always return a constant iterator.
https://wg21.link/p2278r1

[P2321R2] Tim Song. 2021-06-11. zip.
https://wg21.link/p2321r2

[P2374R3] Sy Brand, Michał Dominiak. 2021-12-13. views::cartesian_product.
https://wg21.link/p2374r3

[range-v3] Eric Niebler. range-v3 library.
https://github.com/ericniebler/range-v3